

BlueHarbor Terminal

Documentazione tecnica

Registro operativo per la gestione delle banchine di un terminal container.

Documento rivolto a lettori tecnici (sviluppo, integrazione, manutenzione).

Cliente / Committente	MSC — Mediterranean Shipping Company (scenario didattico)
Prodotto	BlueHarbor Terminal — web app di registro e pianificazione banchine
Versione documento	1.0 — Consegna finale
Data	27 luglio 2026
Team	< inserire nome del team >
Repository	github.com/lufo-336/projectBlueHarbor

Sommario

1. Introduzione

- 1.1 Scopo del documento
- 1.2 Glossario essenziale

2. Panoramica del prodotto

3. Architettura complessiva

- 3.1 Deploy in cloud

4. Componenti principali e responsabilità

- 4.1 Frontend (React)
- 4.2 Backend (ASP.NET Core)
- 4.3 Database

5. Modello dati ad alto livello

6. Regole di dominio e algoritmi

- 6.1 Taglie e compatibilità
- 6.2 Generazione della nave
- 6.3 Ciclo di vita
- 6.4 Algoritmo di accodamento
- 6.5 Modello temporale e Next Day
- 6.6 Manutenzioni banchina
- 6.7 Storico assegnazioni

7. API principali

8. Sicurezza

9. Internazionalizzazione (IT/EN)

10. Decisioni progettuali e compromessi

11. Qualità: build, test e CI/CD

12. Struttura del repository

13. Appendice — accesso rapido

1. Introduzione

1.1 Scopo del documento

Questo documento descrive la struttura e il funzionamento di BlueHarbor Terminal a beneficio di un pubblico tecnico. Copre l'architettura complessiva, i componenti e le loro responsabilità, il modello dati ad alto livello, le regole di dominio con i relativi algoritmi, il contratto delle API, gli aspetti di sicurezza e le principali decisioni progettuali con i compromessi adottati. Il manuale d'uso per l'utente finale è un documento separato.

Contesto. BlueHarbor è realizzato nell'ambito dell'Unità Formativa "Learning by Project". La commessa richiede una piattaforma web interna per registrare le navi in arrivo, pianificare l'uso delle banchine e coordinare il lavoro del personale operativo di un piccolo terminal container. Scenario, nomi e dati sono fittizi e a scopo didattico.

1.2 Glossario essenziale

Termine	Significato
Giorno virtuale	Tempo simulato del sistema (numero intero). Non esiste orario reale: si avanza di un giorno con l'azione "Next Day".
Banchina (berth)	Posto di ormeggio. Il terminal ne ha 8, di taglia fissa. Una banchina ospita solo navi della propria taglia.
Taglia	Dimensione di nave/banchina: S, M, L, XL.
Occupazione	Intervallo di giorni [inizio, fine) in cui una nave occupa una banchina.
Accodamento	Se una banchina è occupata, la nave viene pianificata nel primo slot libero successivo.
Ciclo di vita	Stati della nave: Pending (in attesa) → Assigned (assegnata) → Departed (partita).

2. Panoramica del prodotto

BlueHarbor sostituisce il coordinamento manuale del terminal con una web app a tre ruoli. Il flusso è lineare: l'Operatore registra le navi in arrivo, lo Scheduler le assegna alle banchine compatibili rispettando l'accodamento, e l'avanzamento del tempo ("Next Day") fa concludere le soste. Un ruolo Admin, aggiunto come ruolo di piattaforma, gestisce utenti, manutenzioni e ambiente di simulazione.

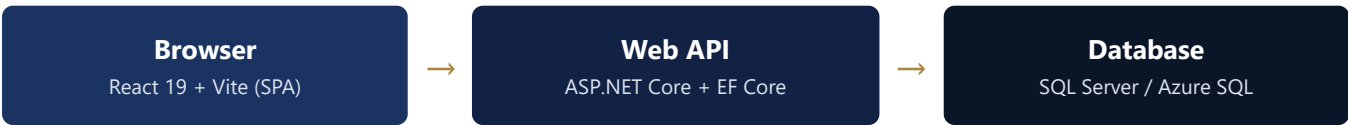
Ruolo	Responsabilità nel dominio
Operatore	Registra nuove navi (il sistema genera taglia, giorno di arrivo e durata; nome e note li inserisce l'operatore). Mantiene lo stato delle navi; può annullare una nave finché è Pending.
Scheduler	Vede le navi in attesa e le assegna alle banchine compatibili secondo le regole (compatibilità di taglia + primo slot libero). Consulta la timeline delle banchine e lo storico assegnazioni.
Admin	Ruolo di piattaforma: gestione utenti, finestre di manutenzione delle banchine, reset della simulazione. Include le viste di Operatore e Scheduler. Non scalca le regole di dominio.

Modello temporale

Il sistema NON è real-time. Mantiene un giorno corrente virtuale; l'azione "Next Day" avanza il tempo di un giorno e imposta a Departed le navi che hanno concluso l'occupazione. Non si gestiscono ore o minuti: è una scelta di dominio della commessa.

3. Architettura complessiva

BlueHarbor è un’applicazione web a tre livelli: un frontend a pagina singola (SPA), una Web API stateless e un database relazionale. Il frontend comunica con il backend solo via HTTP/JSON; il backend è l’unico a parlare con il database.



Frontend — React 19 con Vite. Nessun router: la vista mostrata dipende dal ruolo dell’utente autenticato. Lo stato condiviso è gestito con Context API (autenticazione, giorno virtuale, preferenze UI, notifiche). Tutte le chiamate passano da un unico client HTTP.

Backend — ASP.NET Core Web API con Entity Framework Core. Controller sottili che validano l’input e orchestrano; la logica di dominio pura è isolata in servizi. Gli errori sono restituiti come `Problem Details` (RFC 7807). `GET /api/ships` è paginato e filtrabile.

Database — SQL Server (in locale/container) o Azure SQL (in cloud). Schema gestito con script T-SQL versionati; il modello EF resta allineato e all’avvio `EnsureCreated()` completa schema e seed mancanti.

3.1 Deploy in cloud

In produzione l’app gira su Azure Container Apps con Azure SQL Database (tier Basic). Il deploy è automatico: a ogni merge su main che tocca il progetto, una GitHub Action costruisce l’immagine Docker, la pubblica su GitHub Container Registry (taggata per SHA) e aggiorna la Container App.

Aspetto	Scelta
Hosting applicazione	Azure Container Apps (immagine Docker unica: API + frontend statico)
Database gestito	Azure SQL Database (tier Basic)
Registro immagini	GitHub Container Registry (ghcr.io/lufo-336/blueharbor)
CI/CD	GitHub Actions (.github/workflows/deploy.yml) su push a main
Segreti	Connection string e chiave JWT come secret della Container App, via variabili d’ambiente: non nel repo né nell’immagine
Telemetria	Application Insights, attiva solo se configurata (spenta in locale)

Piano B della demo — un `docker compose up --build` avvia in locale DB + app con un solo comando, con auto-inizializzazione di schema e seed.

4. Componenti principali e responsabilità

4.1 Frontend (React)

Componente	Responsabilità
Viste ruolo (OperatorView, SchedulerView, AdminView)	Interfaccia specifica del ruolo. AdminView incorpora e riusa le viste Operatore e Scheduler tramite tab.
AuthPage	Login, account demo, selettore lingua e tema prima dell'accesso.
Context (Auth, Day, Prefs, Toast)	Stato applicativo: utente/sessione JWT, giorno virtuale reale dal backend, preferenze (tema, formato tempo, lingua), notifiche.
services/api.js	Unico punto di accesso HTTP: allega il token, gestisce 401 (sessione scaduta) e legge il campo detail dei Problem Details.
services/scheduling.js	Replica CLIENT dell'algoritmo di accodamento, usata SOLO per l'anteprima in timeline (la verità resta il server).
services/time.js + i18n/copy.js	Formattazione del tempo locale-aware e dizionario bilingue IT/EN con funzione di traduzione.
Modal, Toast, LoadingSpinner	Componenti UI riutilizzabili.

4.2 Backend (ASP.NET Core)

Componente	Responsabilità
AuthController	Login, /me, logout. Emette il JWT con il claim di ruolo.
ShipsController	CRUD navi, assegnazione, annullo e modifica assegnazione (prima dell'inizio occupazione).
SchedulerController	Dashboard dello Scheduler: banchine, occupazioni, manutenzioni, navi in attesa.
HistoryController	Storico assegnazioni in sola lettura + export CSV.
AdminController	Gestione utenti, manutenzioni banchina, reset della simulazione.
TimeController / SystemController	Giorno virtuale corrente, riepilogo terminal, azione Next Day.
SchedulingRules (servizio)	Funzioni pure: compatibilità di taglia e calcolo del primo slot libero (accodamento).
TimeService	Avanzamento transazionale del giorno virtuale e transizione a Departed.
TokenService	Firma/validazione JWT.
ShipGeneratorService	Generazione casuale dei dati nave (taglia, arrivo, durata) secondo la commessa.

4.3 Database

Schema relazionale con le 8 banchine fisse seminate all'avvio (1 XL, 1 L, 2 M, 4 S), una tabella di impostazioni per il giorno virtuale e lo storico append-only. Gli script incrementali (script5→script8) documentano l'evoluzione: base, storico, ruolo Admin, manutenzioni banchina.

5. Modello dati ad alto livello

Le entità principali e i loro campi salienti. Le chiavi esterne collegano le navi alle banchine e lo storico a navi e banchine.

Entità	Campi principali	Note
User	Id, Username (email), Role, PasswordHash, IsActive	Ruolo ∈ {Operator, Scheduler, Admin} (CHECK). Un solo ruolo per utente. Hash PBKDF2.
Ship	Id, Name, Size, ArrivalDay, Duration, Status, BerthId?, OccupationStartDay?, Notes	Size ∈ {S,M,L,XL}; Status ∈ {Pending,Assigned,Departed}. BerthId/OccupationStartDay valorizzati all'assegnazione.
Berth	Id, Name, Size	Insieme FISSO di 8: XL-1, L-1, M-1, M-2, S-1..S-4.
BerthMaintenance	Id, BerthId, StartDay, EndDay	Finestra [inizio, fine) di indisponibilità della banchina.
AssignmentHistory	Id, ShipId, BerthId, EventType, OccupationStart/EndDay, EventDay (+ snapshot)	Tabella append-only. EventType ∈ {Assigned, Departed}. Alimentata da assegnazione e Next Day.
Setting	Key, Value (CurrentVirtualDay, Day1Date)	Stato di sistema: giorno virtuale corrente ed eventuale data calendario del giorno 1.

Storico append-only

AssignmentHistory non viene mai modificato o cancellato: è un audit trail. Le righe sono scritte nella STESSA transazione dell'assegnazione e della partenza, garantendo coerenza tra stato della nave e cronologia.

6. Regole di dominio e algoritmi

6.1 Taglie e compatibilità

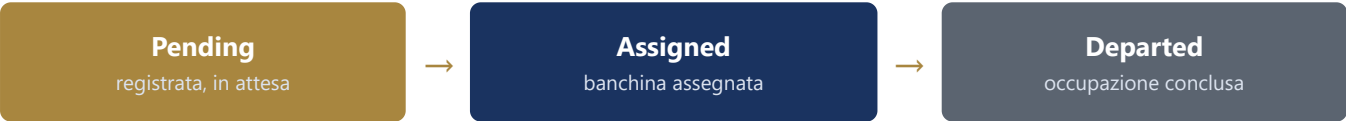
Regola. Una banchina ospita solo navi della propria identica taglia (`shipSize == berthSize`). Non esistono compatibilità "una taglia più grande": è un vincolo di dominio esplicito.

6.2 Generazione della nave

Alla creazione, il sistema genera automaticamente i dati operativi; l'operatore fornisce solo nome e note. La nave nasce in stato Pending, senza banchina.

Campo	Regola di generazione
Taglia	Casuale tra S, M, L, XL (uniforme).
Giorno di arrivo	Casuale da +1 a +30 giorni rispetto al giorno corrente.
Durata occupazione	Casuale tra 3 e 15 giorni (inclusi).

6.3 Ciclo di vita



6.4 Algoritmo di accodamento

Obiettivo. Dato l'arrivo, la durata, il giorno corrente e le occupazioni già presenti su una banchina, trovare il PRIMO giorno con un intervallo libero lungo almeno la durata, mai prima dell'arrivo né nel passato. È il cuore del prodotto e vive come funzione pura in `SchedulingRules.ComputeOccupationStartDay`.

```
candidate = max(arrivalDay, currentDay)
```

- Si scorrono le occupazioni esistenti in ordine di inizio crescente.
- Se prima della prossima occupazione c'è un buco $\geq$ durata ($\text{occ.start} \geq \text{candidate} + \text{durata}$) → il candidato va bene: è il giorno di inizio.
- Altrimenti l'occupazione blocca: si sposta il candidato alla sua fine ($\text{candidate} = \max(\text{candidate}, \text{occ.end})$) e si continua.
- Se nessuna occupazione successiva blocca, la nave si accoda in fondo a partire dal candidato.

Anteprima lato client

Lo stesso algoritmo è replicato nel frontend (`services/scheduling.js`) SOLO per mostrare in tempo reale l'anteprima del primo slot libero mentre lo Scheduler valuta una banchina. La decisione VERA è sempre quella calcolata dal server. Un controllo di parità (`checks/scheduling.check.mjs`) tiene allineate le due copie.

6.5 Modello temporale e Next Day

L'azione Next Day avanza il giorno virtuale di uno, in modo transazionale, e imposta a Departed le navi la cui occupazione è terminata. Non effettua assegnazioni automatiche. Le viste si aggiornano di conseguenza.

6.6 Manutenzioni banchina

L'Admin può dichiarare finestre [inizio, fine) in cui una banchina non è utilizzabile. Le navi assegnate DOPO si accodano oltre la finestra, riusando lo stesso algoritmo (la manutenzione blocca come un'occupazione). Una manutenzione che incrocia navi GIÀ assegnate viene rifiutata (409): le navi assegnate non si spostano mai. Il set di 8 banchine non cambia: cambia solo la loro disponibilità.

6.7 Storico assegnazioni

Ogni assegnazione e ogni partenza generano una riga in `AssignmentHistory`, consultabile in sola lettura (con filtro per tipo evento) ed esportabile in CSV. È un registro di ciò che è accaduto, non un meccanismo di riassegnazione.

7. API principali

Autenticazione. Tutti gli endpoint applicativi richiedono un JWT (`Authorization: Bearer ...`) e sono protetti per ruolo (`[Authorize(Roles=...)]`). L'accesso non autorizzato restituisce 401/403. Gli errori funzionali sono Problem Details con un campo `detail` leggibile.

Area	Endpoint (metodo)	Ruolo
Auth	POST /api/auth/login · GET /api/auth/me · POST /api/auth/logout	Pubblico / autenticato
Navi	GET /api/ships (paginato, filtri) · POST · PUT · DELETE /api/ships/{id}	Operator / Admin
Assegnazione	POST /api/ships/{id}/assign · POST /unassign · PUT /assignment	Scheduler / Admin
Scheduler	GET /api/scheduler/dashboard	Scheduler / Admin
Storico	GET /api/history · GET /api/history/export (CSV)	Scheduler / Admin
Tempo/Sistema	GET /api/system/current-day · GET /api/system/summary · POST /api/time/next-day	Autenticato
Admin utenti	GET/POST /api/admin/users · PUT/DELETE /api/admin/users/{id}	Admin
Admin manutenzioni	GET/POST /api/admin/maintenance · DELETE /api/admin/maintenance/{id}	Admin
Admin simulazione	POST /api/admin/simulation/reset	Admin

8. Sicurezza

- **Autenticazione JWT.** Al login il server emette un token firmato col claim di ruolo; il frontend lo allega a ogni chiamata. Alla scadenza (401) la sessione viene invalidata e si torna al login.
- **Autorizzazione per ruolo.** Ogni endpoint dichiara i ruoli ammessi; dominio e piattaforma restano separati (l'Admin non assegna navi al posto dello Scheduler).
- **Password.** Memorizzate come hash PBKDF2 (con salt), mai in chiaro.
- **Segreti.** Connection string e chiave JWT come variabili d'ambiente/secret della piattaforma cloud: fuori dal repository e dall'immagine.
- **Superficie d'errore.** Un gestore globale converte le eccezioni impreviste in un Problem Details generico, senza esporre dettagli interni.

9. Internazionalizzazione (IT/EN)

L'interfaccia è completamente bilingue italiano/inglese, con selettore in navbar e nella pagina di login. L'impostazione è persistente e la lingua iniziale è dedotta dal browser (fallback italiano).

- Dizionario unico { it, en } con funzione di traduzione `t()`; i testi dinamici usano funzioni con parametri.
- Formattazione locale-aware: date (it-IT / en-GB), durate ("giorni"/"days") e prefisso del giorno virtuale (g/d).
- I messaggi d'errore del backend (in italiano) sono tradotti lato client con una mappa dedicata (voci esatte + pattern per i messaggi dinamici) e fallback morbido all'italiano.

10. Decisioni progettuali e compromessi

Decisione	Motivazione / compromesso
EF Core al posto di ADO.NET	Deviazione consapevole dal piano iniziale: evita di riscrivere il layer dati a mano, a fronte di un minor controllo sul SQL generato.
Admin come ruolo di piattaforma	Estensione ammessa e giustificata: gestisce utenti/ambiente ma NON scavalca le regole (niente riassegnazioni, niente bypass). Mantiene "un ruolo per utente".
Annulla nave solo se Pending	La commessa vieta modifiche DOPO l'assegnazione, non prima: correggere una registrazione errata è lecito ed è dichiarato come assunzione.
Manutenzioni banchina	Cambiano la disponibilità, non il set fisso di banchine; riusano l'algoritmo di accodamento; non spostano navi già assegnate.
Replica client dell'algoritmo	Solo per l'anteprima immediata in timeline; la verità resta il server, con check di parità.
Niente auto-scheduling / KPI / real-time	Fuori scope esplicito: si privilegia correttezza, chiarezza e qualità architeturale.

11. Qualità: build, test e CI/CD

- **Avvio locale.** Docker Compose (un comando, con seed automatico) oppure server separati (dotnet run + npm run dev con proxy Vite).
- **Test.** dotnet test esegue gli unit test xUnit su SchedulingRules (accodamento) e sull'hashing delle password.
- **Parità algoritmo.** Il check node checks/scheduling.check.mjs verifica che la replica client resti allineata al backend.
- **CI/CD.** GitHub Actions: build immagine → push su GHCR → aggiornamento della Container App, a ogni merge su main.
- **Schema versionato.** Script T-SQL incrementali + modello EF allineato; EnsureCreated() completa lo schema mancante all'avvio.

12. Struttura del repository

Cartella	Contenuto
BlueHarbor_QPD_WSA.Server/	Backend: controller, servizi di dominio (SchedulingRules, TimeService...), auth, modelli EF.
BlueHarbor_QPD_WSA.Server.Tests/	Unit test xUnit.
frontend/	Frontend React: viste, context, services, i18n.
database/	Script T-SQL incrementali (script5→script8).
docs/	Documentazione di progetto e di consegna.
.github/workflows/	Pipeline CI/CD (deploy.yml).

13. Appendice — accesso rapido

URL pubblico (cloud): <https://app-blueharbor.politeriver-97b0d932.swedencentral.azurecontainerapps.io>

Ruolo	Email	Password
Operatore	operator@blueharbor	operator123
Scheduler	scheduler@blueharbor	scheduler123
Admin	admin@blueharbor	admin123

Le credenziali demo sono a scopo di valutazione. In un'installazione reale vanno sostituite.